ARE 212 Section 13:
Spatial Data

Fiona Burlig

April 21, 2016

Increasingly, applied econometricians are embedding spatial data into our analyses. A whole host of research questions and designs become possible if you are able to take advantage of and work with the huge volume of spatial data that is rapidly becoming available.[1] That said, working with spatial data takes a few new tricks.[2] We'll first go through a variety of spatial data types, then show off the long-promised (and dramatically over-used) nighttime lights dataset. Coooooool.

Off we go

First off, we often make a big deal out of spatial data - but it's just data like anything else. Anybody who tries to tell you that spatial data is *completely different* (!) from anything you've seen before is trying to sell you a textbook. The main difference between spatial datasets and any other type of dataset is that spatial data usually come in 2 (or even 3) dimensions. For practical purposes, this usually means that a spatial dataset has latitude and longitude variables in some form or another.

To deal with spatial data, we'll need a bunch of new packages: `GISTools`, `rgdal`, `rgeos`, `maptools`, `raster` (all spatial utilities), and `broom` (this will let us turn spatial data into a format that `ggplot2` can handle). We'll also get a new package that will let us more easily deal with dates (because why not): `lubridate`, and a final one which gives us access to a bunch of new color palettes: `RColorBrewer`. The number of packages we're installing here should make you uncomfortable. While you certainly *can* do spatial data analysis in `R`, it's definitely a challenge compared with some other options out there (`Matlab` and ArcGIS, for example).[3] The benefits are that you'll have a completely integrated workflow, all in one program; and that once you've wrangled your spatial data to turn it into something useful, `R` is great for everything else. The big problem with doing spatial data in `R` is that there's no unified spatial toolkit, meaning that different data types and formats and functions don't necessarily get along very well. I've tried to make this as headache-free as possible, but just be aware that (in my opinion) spatial data work is not necessarily `R`'s comparative advantage.

```
install.packages('GISTools')
install.packages('rgdal')
install.packages('rgeos')
install.packages('raster')
install.packages('maptools')
```

[1] On top of that, if you work with spatial data, you can make pretty maps - which, as Max likes to say, is the easiest way to make it look like you've done a lot of work.

[2] We actually already saw one of these a few weeks ago: Conley HAC standard errors.

[3] Major downside of these programs is that they're not free - but UC Berkeley pays for them, so while you're here, you can get access.

```
install.packages('broom')
install.packages('lubridate')
install.packages('RColorBrewer')
```

And we'll grab our usual setup. Note that the order in which you load these packages is important. I've set things up such that they'll work; *caveat econometricus* if you change this yourself:

```
library(raster)
library(GISTools)
library(readr)
library(dplyr)
library(lubridate)
library(xtable)
library(ggplot2)
library(rgeos)
library(rgdal)
library(maptools)
library(broom)

###### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbldfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

OLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  # beta
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  #std error
  se <- sqrt(s2 * diag(XpXinv))
  # t statistic & its pvalue
  tStat <- (betahat - 0) / se
  pVal <-  tStat %>%
```

```
    abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*"(2)
  olsOut <- list(X, betahat, se, tStat, pVal) %>%
    as.tbl_df()
  names(olsOut) <- c("variable", "betahat", "SE", "tStat", "tStatPVal")
  return(olsOut)
}

########## RANDOMIZATION SEED
set.seed(12345)

########## GGPLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
        panel.border = element_rect(fill = NA, color = "black"),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_line(color = "gray5"),
        axis.text = element_text(color = "black", size = 10),
        axis.title = element_text(color = "black", size = 12),
        legend.key = element_blank()
        )
```

What the frack?

Now we can get to the actual spatial stuff. We're going to use some georeferenced data to think about unconventional oil and gas drilling in Pennsylvania.[4] We'll start with a spatial data file with the shape of each county in the state.[5] In keeping with the ultra-creative naming conventions we've seen all semester, this data comes in *shapefile* format. This is the main format used by ArcGIS, so you'll see a lot of data that comes packaged this way. Bringing it into `R` is easy:

```
counties <- readOGR(dsn=getwd(), layer = "PA_counties")

## OGR data source with driver: ESRI Shapefile
## Source: "D:/Fiona/Dropbox/UC Berkeley/2015-2016/ARE 212/Section 13", layer: "PA_counties"
## with 67 features
## It has 13 fields
```

What's in this object? Normally, we'd look at it by just printing it. This will be gross here, because this shapefile contains a ton of information. This particular file contains *polygons* - outlines of shapes. We can make sure that it's read correctly into `R` by checking its class:

[4] Why Pennsylvania? There's a big shale formation, the Marcellus Shale, that runs through the Western part of the state, which makes it a good place to look at fracking. There's also a surprisingly large amount of data available.

[5] These data are available from the US Census Bureau.

```
class(counties)

## [1] "SpatialPolygonsDataFrame"
## attr(,"package")
## [1] "sp"
```

Perfect. This is R's version of a polygon shapefile. What's actually in this dataset? We can figure this out by looking at its `slots`. Just like calling `names()` on an object can tell us something about its contents, `slotNames()` can do this too, for more complex objects:

```
slotNames(counties)

## [1] "data"        "polygons"    "plotOrder"   "bbox"
## [5] "proj4string"
```

This is cool! Our shapefile contains a bunch of interesting pieces. Let's walk through them quickly. Most importantly, we've got `polygons`. This is the spatial component of our spatial dataset. Each polygon is essentially a long two-variable dataframe with longitudes (x) and latitudes (y). Think of the polygon piece of a shapefile like a giant connect-the-dots.

Our shapefile also contains `data`. The dataset attached to a shapefile is just like a regular dataframe, with the additional cool feature that each row is actually associated with one of the polygons in our shapefile. Let's take a look at the data that comes with the Pennsylvania county shapefile (we'll rename our names to all be lower case as well, while we're at it):

```
head(counties@data, 5)

##   STATEFPEC COUNTYFPEC CNTYIDFPEC  NAMEEC     NAMELSADEC LSADEC
## 0        42        001      42001   Adams   Adams County     06
## 1        42        027      42027  Centre  Centre County     06
## 2        42        023      42023 Cameron Cameron County     06
## 3        42        025      42025  Carbon  Carbon County     06
## 4        42        029      42029 Chester Chester County     06
##   CLASSFPEC MTFCCEC FUNCSTATEC    ALANDEC AWATEREC  INTPTLATEC
## 0        H1   G4020          A 1342811635  8560860 +39.8694707
## 1        H1   G4020          A 2147483647  7879260 +40.9091673
## 2        H1   G4020          A 1026698470  5664145 +41.4382882
## 3        H1   G4020          A  988299800 15353755 +40.9178324
## 4        H1   G4020          A 1943960881 22601992 +39.9737772
##     INTPTLONEC
## 0 -077.2177295
## 1 -077.8478195
## 2 -078.1983134
## 3 -075.7094276
## 4 -075.7503816

names(counties@data) <- tolower(names(counties@data))
```

This dataset is relatively uninteresting - it just contains a bunch of identifying information about each county (but not much else in terms of demographics or other things we might be interested in). We'll have to bring something else in ourselves (we'll get there).

Also contained in our shapefile: the `bbox` and `plotOrder`. They're largely irrelevant for our purposes - they tell the base plotting commands how to display the data. We've also got a *projection*: `proj4string`. This tells `R` both how to display and how to "think about" our shapefile. Remember that the world is round. We're trying to represent it with a data object on our (flat) computer screens. The projection defines what dimensions to stretch to make this representation happen. What does it look like? Let's look! Just like we can grab a named object with the `$` operator, we can grab a named slot with the operator.

```
counties@proj4string

## CRS arguments:
##  +proj=longlat +datum=NAD83 +no_defs +ellps=GRS80
## +towgs84=0,0,0
```

This tells us that we're using latitude and longitude to identify our data, and that we're using the North American Datum 83 (a common choice for looking at the US). Since we're dealing with a relatively small geographic area, this won't affect us too much - but you should be aware of what projection you're using. In particular, if you're going to combine multiple geographic datasets, it's important that they all use the same projection.[6]

Ooh, pretty!

Okay, now that we've had a quick introduction to our data, let's plot our shapefile. Since a polygon is just a big collection of longitude and latitude values, we can just plot it like anything else in `ggplot2`. The first thing we have to do is to convert our polygon into a `tbl_df` that `ggplot2` can handle, using `broom`'s `tidy()` function. In the mean time, we'll merge, or `join()`, the additional data that came with the shapefile, into this new `tbl_df`. We'll keep doing this, so we'll write a function to take care of it for us:

```
mapToDF <- function(shapefile) {
  # first assign an identifier to the main dataset
  shapefile@data$id <- rownames(shapefile@data)
  # now ``tidy'' our data to convert it into
  #  dataframe that's usable by ggplot2
  #  the region command keeps polygons together
  mapDF <- tidy(shapefile) %>%
      # and this data onto the information attached to the shapefile
    left_join(., shapefile@data, by = "id") %>%
    as.tbl_df()
  return(mapDF)
}


paCounties <- mapToDF(counties)
```

[6] The most common global projection is WGS84, so keep an eye out for that as well.

```
## Regions defined for each Polygons

# take a look at this
paCounties

## Source: local data frame [175,567 x 20]
##
##         long      lat order  hole  piece  group    id statefpec
##        (dbl)    (dbl) (int) (lgl) (fctr) (fctr) (chr)    (fctr)
## 1  -76.99950 39.79106     1 FALSE      1    0.1     0        42
## 2  -76.99943 39.79044     2 FALSE      1    0.1     0        42
## 3  -76.99939 39.79013     3 FALSE      1    0.1     0        42
## 4  -76.99939 39.79003     4 FALSE      1    0.1     0        42
## 5  -76.99931 39.78907     5 FALSE      1    0.1     0        42
## 6  -76.99929 39.78852     6 FALSE      1    0.1     0        42
## 7  -76.99929 39.78846     7 FALSE      1    0.1     0        42
## 8  -76.99929 39.78845     8 FALSE      1    0.1     0        42
## 9  -76.99940 39.78356     9 FALSE      1    0.1     0        42
## 10 -76.99940 39.78263    10 FALSE      1    0.1     0        42
## ..       ...      ...   ...   ...    ...    ...   ...       ...
## Variables not shown: countyfpec (fctr), cntyidfpec (fctr),
##   nameec (fctr), namelsadec (fctr), lsadec (fctr), classfpec
##   (fctr), mtfccec (fctr), funcstatec (fctr), alandec (int),
##   awaterec (int), intptlatec (fctr), intptlonec (fctr)
```

Now that we've got this dataset, it's easy to plot using `ggplot2`[7]. For maps, I actually prefer to remove the boundary around my plot, so let's update `myThemeStuff` accordingly:

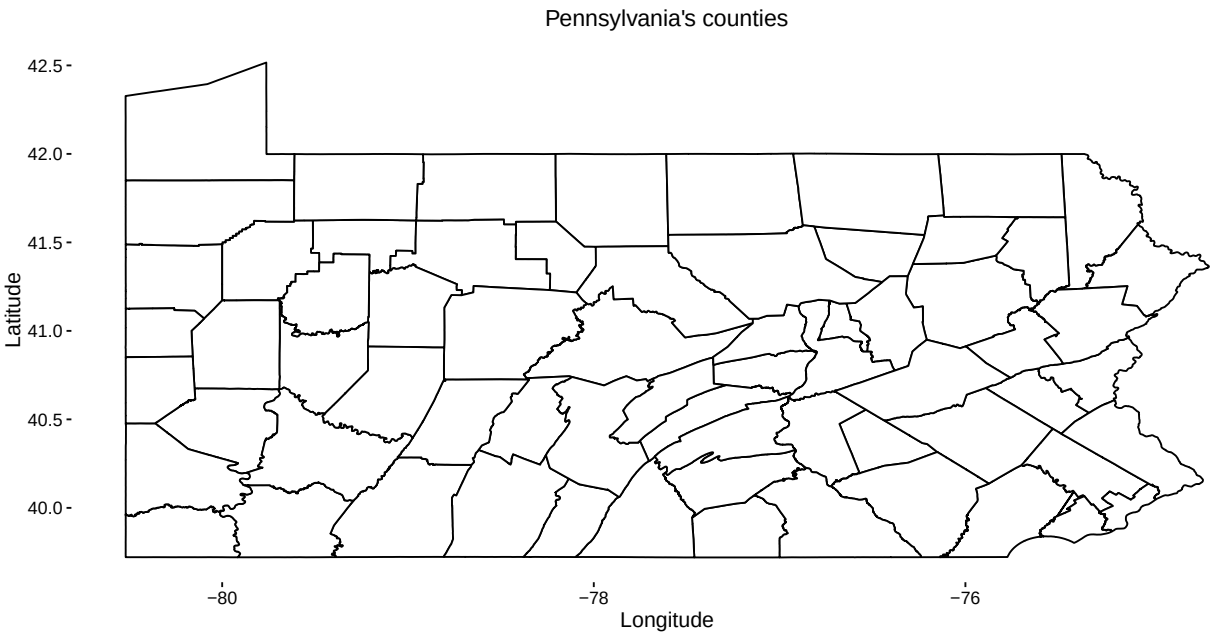
```
myMapThemeStuff <- theme(panel.background = element_rect(fill = NA),
        panel.border = element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_line(color = "gray5"),
        axis.text = element_text(color = "black", size = 10),
        axis.title = element_text(color = "black", size = 12),
        legend.key = element_blank()
        )
```

Okay, here we go.

```
paMap <- ggplot(data = paCounties, aes(x = long, y = lat, group =  id)) +
  geom_polygon(color = 'black', fill = 'white') +
  myMapThemeStuff + ggtitle("Pennsylvania's counties") +
  xlab("Longitude") + ylab("Latitude")
```

[7] Or, on obnoxious data science parlance, "visualize."

```
paMap
```



Woohoo! A map! We must've done a lot of work. Just a map by itself isn't very interesting, though. Let's bring in another dataset, containing the locations of every unconventional well drilled in Pennsylvania between 2002 and 2013.[8] We'll do this with a familiar command:

```
wells <- read.csv("PA_wells.csv") %>%
  # convert back to a dataframe because
  #  coordinates() doesn't like tbl_dfs (boo)
  as.data.frame()
names(wells) <- tolower(names(wells))
head(wells)

##   spud_date       api   ogo_num                         operator
## 1 5/24/2002 125-22033 OGO-49020              BELDEN & BLAKE CORP
## 2 5/31/2003 125-22074 OGO-60915 RANGE RESOURCES APPALACHIA LLC
## 3 6/14/2003 063-33347 OGO-38958                  XTO ENERGY INC
## 4 8/27/2003 129-25004 OGO-60915 RANGE RESOURCES APPALACHIA LLC
## 5  9/6/2003 129-25012 OGO-60915 RANGE RESOURCES APPALACHIA LLC
## 6 2/27/2004 129-25209 OGO-33810             GREAT OAK ENERGY INC
##                  region       county         municipality
## 1 EP DOGO SWDO Dstr Off   Washington          Hanover Twp
## 2 EP DOGO SWDO Dstr Off   Washington   Mount Pleasant Twp
## 3 EP DOGO SWDO Dstr Off      Indiana            Grant Twp
## 4 EP DOGO SWDO Dstr Off Westmoreland South Huntingdon Twp
## 5 EP DOGO SWDO Dstr Off Westmoreland South Huntingdon Twp
## 6 EP DOGO SWDO Dstr Off Westmoreland            Salem Twp
```

[8] This comes to us from the friendly folks over at the FracTracker Alliance.

```
##         farm_name well_code_desc                well_status
## 1 ANDERSON UNIT 1            GAS                     Active
## 2          RENZ 1            GAS                     Active
## 3 LEONARD MUMAU 2            GAS                     Active
## 4 HEPLER CALVIN 1  COMB. OIL&GAS Regulatory Inactive Status
## 5       STAHL E 1  COMB. OIL&GAS                     Active
## 6   EXPORT FUEL 1            GAS                     Active
##   latitude longitude configuration unconventional
## 1 40.46467 -80.47789 Vertical Well            Yes
## 2 40.28316 -80.28402 Vertical Well            Yes
## 3 40.80562 -78.93993 Vertical Well            Yes
## 4 40.15899 -79.70181 Vertical Well            Yes
## 5 40.15602 -79.72340 Vertical Well            Yes
## 6 40.43544 -79.58450 Vertical Well            Yes
```

Notice that this dataset includes a latitude and a longitude variable - this means it's spatial. Before we plot it, though, we need to make sure that it'll use the same projection as our our county polygon. It's currently a `tbl_df` - let's turn it into a `SpatialPointsDataFrame` (like the polygon, except for points, obviously). In doing so, we'll attach a projection.

```
coordinates(wells) <-~longitude + latitude
class(wells)

## [1] "SpatialPointsDataFrame"
## attr(,"package")
## [1] "sp"
```

This thing is now a `SpatialPointsDataFrame` - did it automatically get a projection?

```
proj4string(wells)

## [1] NA
```

No! Boo. We have to assign it ourselves. The bad news is that I'm not sure exactly what the original projection was (it wasn't labeled on the download website). We'll guess that it's WGS84, and then re-project it to match our county data's NAD83.[9] No problem:

```
# assign a projection (WGS84)
proj4string(wells) <- CRS("+proj=longlat +datum=WGS84")
# re-project this to match the county data
wells <- spTransform(wells, CRS(proj4string(counties)))
#check
proj4string(wells)

## [1] "+proj=longlat +datum=NAD83 +no_defs +ellps=GRS80 +towgs84=0,0,0"
```

Now that the projection is good, we'll convert our dataset back to `tbl_df` format so that `ggplot2` can handle it.

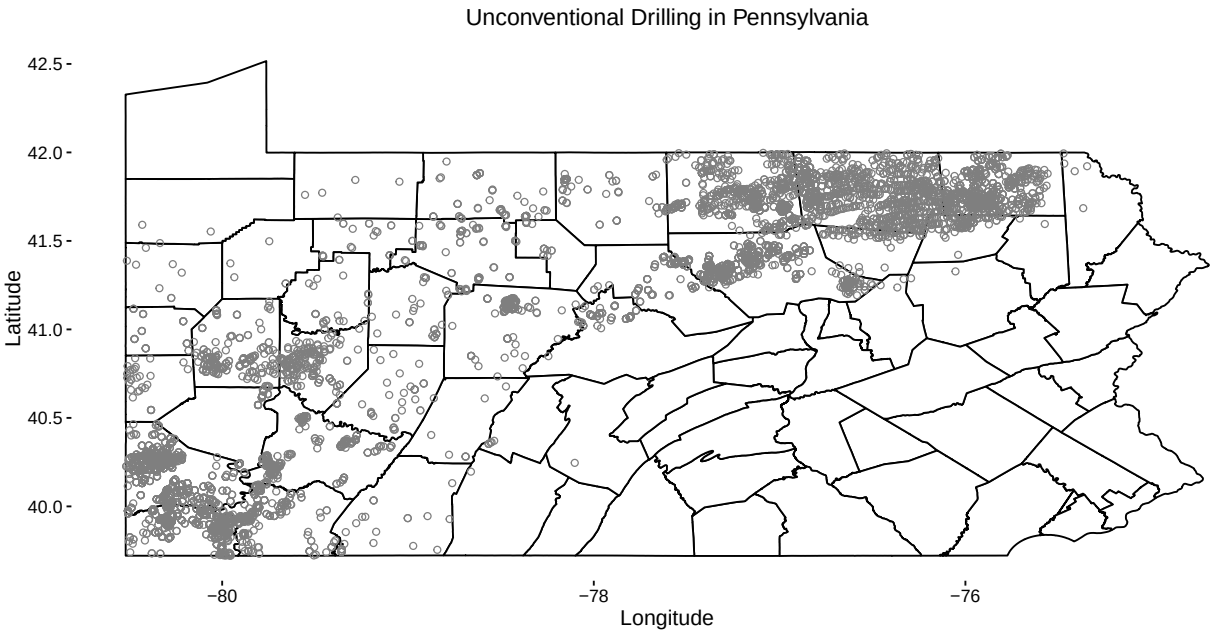
[9] Why do this? So that you can see how to do it in the future if you need to. Hashtag teachable moments.

```
wellsDF <- as.tbl_df(wells)
```

Now we'll plot both the counties and the wells on our map:

```
paMap <- ggplot() +
  geom_polygon(data = paCounties, aes(x = long, y = lat, group =  id),
                                      color = 'black', fill = 'white') +
  geom_point(data = wellsDF, aes(x = longitude, y = latitude),
             shape = 21, color = 'gray50') +
  myMapThemeStuff + ggtitle("Unconventional Drilling in Pennsylvania") +
  xlab("Longitude") + ylab("Latitude")
```

```
paMap
```



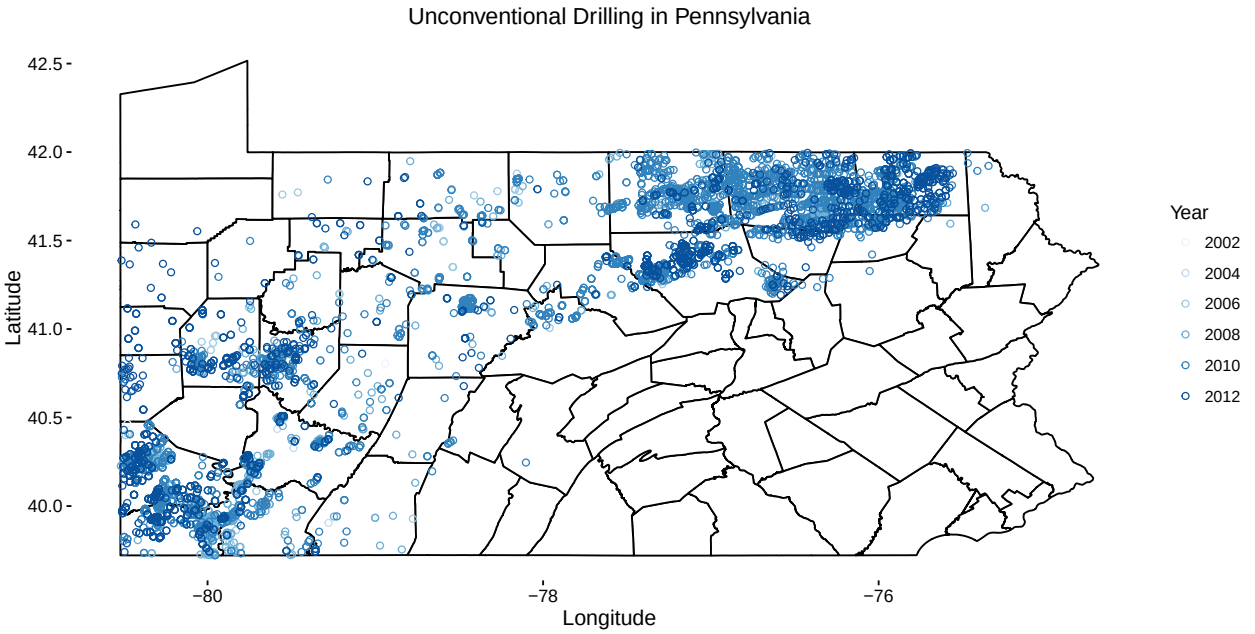
If we want, we can also plot different colors by year of well drilling. To do this, we'll first convert the spud date (drill date) variable to date format, extract the year, and convert this into a factor. In order to make this not impossible to look at, let's actually make pairs of years:

```
wellsDF <- mutate(wellsDF, date = mdy(spud_date),
                  year = year(date)) %>%
  mutate(year = 2*(floor(year / 2)))
wellsDF <- mutate(wellsDF, year = as.factor(year))
```

This is also a great excuse to bring in one of my favorite `R` packages: `RColorBrewer`.[10]

```
paMap <- ggplot() +
  geom_polygon(data = paCounties,
               aes(x = long, y = lat, group =  id),
               color = 'black', fill = 'white') +
  geom_point(data = wellsDF, aes(x = longitude, y = latitude, color = year),
             shape = 21) +
  scale_color_brewer(palette="Blues") +
  myMapThemeStuff + ggtitle("Unconventional Drilling in Pennsylvania") +
  xlab("Longitude") + ylab("Latitude") +labs(color = "Year")
```

```
paMap
```



[10] If you're interested in seeing all of the available color palettes from this great package, just type `display.brewer.all()` into your console.

Playing around

Whoa. Lots of recent wells! One other thing that's immediately obvious from this figure is that conventional drilling *isn't* happening all over the state. Instead, it's constrained to the northwest region. Why is this? Let's bring in some new data, which will make this clear. We're going to grab a shapefile of the Marcellus shale play - the rock formation from which you can actually extract hydrocarbons.[11]

```
playBdry <- readOGR(dsn=getwd(),
                        layer = "ShalePlay_Marcellus_Boundary_EIA_Aug2015_v2")

## OGR data source with driver: ESRI Shapefile
## Source: "D:/Fiona/Dropbox/UC Berkeley/2015-2016/ARE 212/Section 13", layer: "ShalePlay_Marcellus_B
## with 1 features
## It has 7 fields
```

And we'll check the projection:

```
playBdry@proj4string

## CRS arguments:
##  +proj=longlat +datum=WGS84 +no_defs +ellps=WGS84
## +towgs84=0,0,0
```

Whoops - this is WGS84. We'll have to convert it:

```
playBdry <- spTransform(playBdry, CRS(proj4string(counties)))
```

Easy-peasy lemon squeezy. Let's prepare these shapefiles for plotting using our `mapToDF()` function from earlier to convert this into a dataframe:

```
bdryDF <- mapToDF(playBdry)

## Regions defined for each Polygons
```

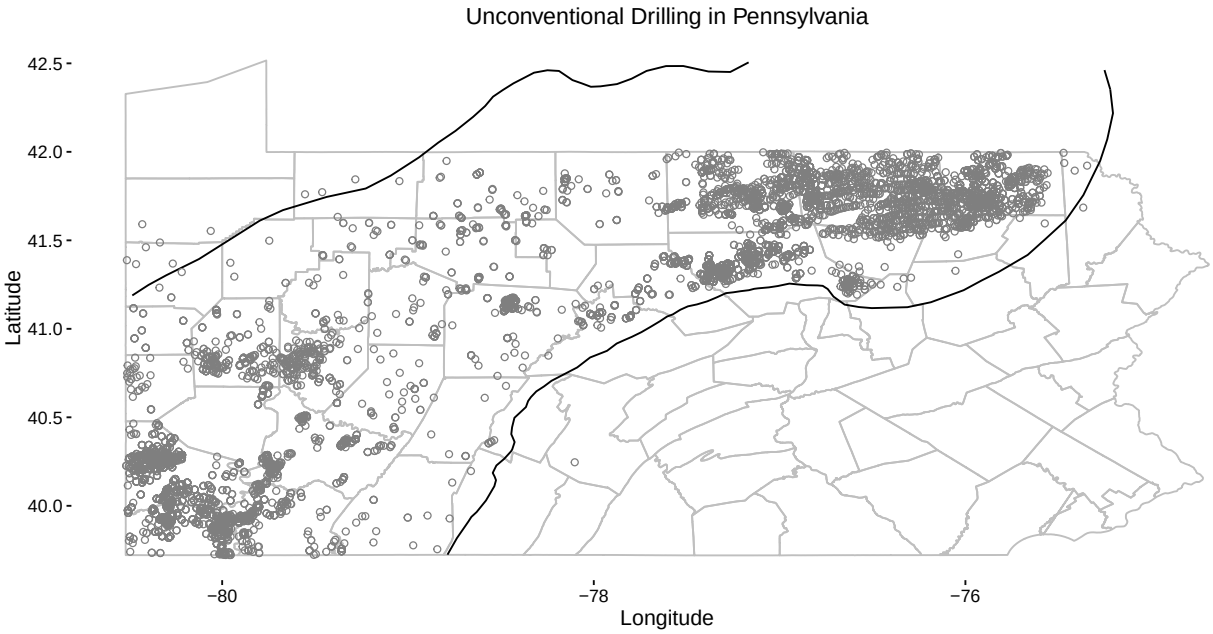
Now we can easily plot these things:

```
bigPlot <- ggplot(data = bdryDF, aes(x = long, y = lat)) +
  geom_polygon(data = paCounties, aes(x = long, y= lat, group = id),
               color = 'gray75', fill = 'NA') +
  geom_path(data = bdryDF, aes(x = long, y = lat),
               color = 'black') +
  geom_point(data = wellsDF, aes(x = longitude, y = latitude),
             color = "gray50", shape = 21) +
  # put in a bounding box to restrict ourselves to the
  #  part of the play in PA
  xlim(counties@bbox[1,1], counties@bbox[1, 2]) +
  ylim(counties@bbox[2,1], counties@bbox[2, 2]) +
```

[11] This dataset comes from the EIA.

```
  ggtitle("Unconventional Drilling in Pennsylvania") +
  xlab("Longitude") + ylab("Latitude") +
  myMapThemeStuff
```

```
bigPlot
```



Nearly all of our wells are within the play - which makes sense, of course. Even within the play, though, there seems to be massive heterogeneity in well locations. Suppose we're interested in seeing whether there's any correlation between a county's demographics and its well count. To do this, we'll have to count the number of wells in each county. We'll also need some demographic data.

We can count wells using the data we already have loaded, so let's start there. We'll do this using the `poly.counts()` function from the `GISTools` package.[12]

```
# return the number of wells in a county
wellsInCty <- poly.counts(wells, counties) %>%
  as.tbl_df() %>%
  mutate(id = rownames(counties@data))

names(wellsInCty) <- c("wells", "id")

wellsInCty <- mutate(wellsInCty,
                     wells = ifelse(is.na(wells) == TRUE, 0, wells))
```

Since this is a spatial object, after all, let's go ahead and plot it. To do this, we'll `left_join()` our new column into our county dataframe:

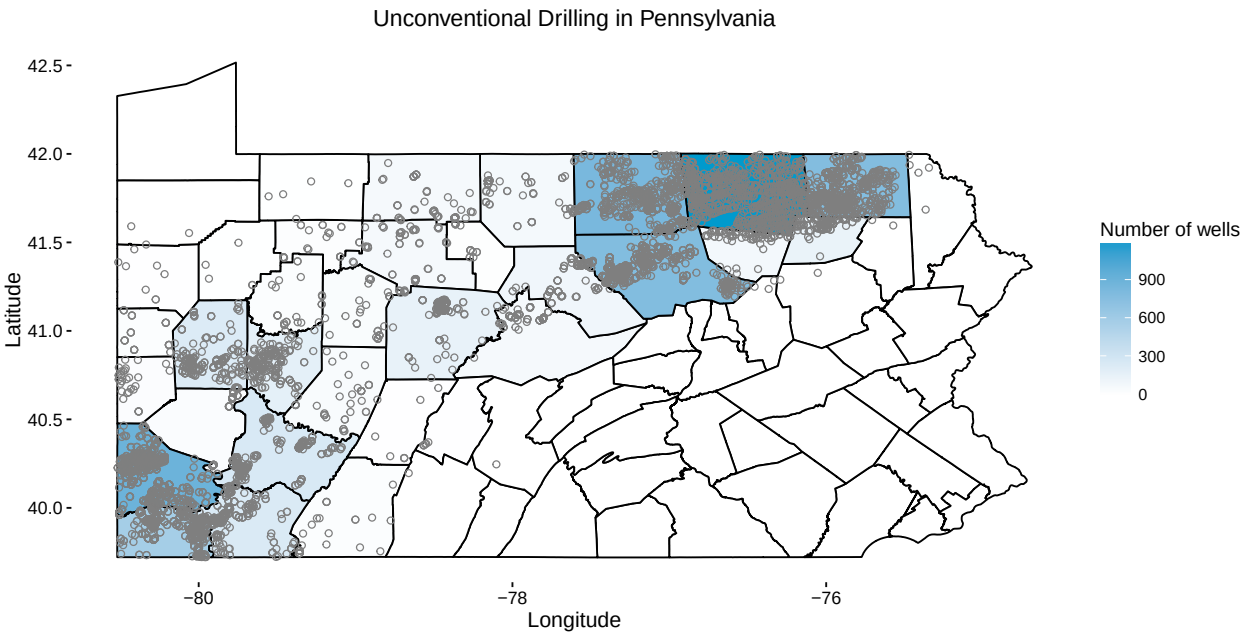
```
paCounties <- left_join(paCounties, wellsInCty, by = "id")
```

And plot:

```
countyPlot <- ggplot() +
  geom_polygon(data = paCounties, aes(x = long, y = lat, group = id, fill = wells), color = 'black')
    geom_point(data = wellsDF, aes(x = longitude, y = latitude),
               color = "gray50", shape = 21) +
  ggtitle("Unconventional Drilling in Pennsylvania") +
  xlab("Longitude") + ylab("Latitude") +
  scale_fill_gradient(low = "white", high = "deepskyblue3") +
  labs(fill = "Number of wells") +
  myMapThemeStuff
```

[12] Be very careful when you do this - and make sure to plot your results! It's easy to end up with data that gets re-ordered in the counting process, as I discovered in the first version of these notes using `sp`'s `over()` function. It's always good to do a sanity check and make sure the output looks right.

```
countyPlot
```



Regressions...in spaaaaace!

To do actual analysis, we want to merge this (hard-won) column into our county data.[13] But our county data are currently a giant huge dataframe that's kind of unweildy, because there's one row per lat-long combination. To make things easier for ourselves, let's grab the *unique* county data from the original shapefile:

```
countyData <- counties@data %>%
  as.tbl_df() %>%
  mutate(id = rownames(counties@data)) %>%
  # for easier merging later
  rename(fips = countyfpec)
```

Much more manageable. We're actually going to want to bring in some new demographic data for our analysis; all we need out of this county dataset is a way to link our well counts to our other dataset. Let's merge the `county` variable from our `countyData` dataset into our well counts, and only keep the few variables that we need:

```
countyWells <- left_join(wellsInCty, countyData, by="id") %>%
  select(fips, id, wells)
```

Next, let's bring in our (long-promised) demographic data:

```
countyDemogs <- read_csv("PA_county_data.csv") %>%
  # remove the row for the whole state
  filter(NAME != "Pennsylvania") %>%
  select(COUNTY, NAME, `Total Population`,
```

[13] Apologies for making the same joke in section like twelve times.

```
         `Median Age`, `Average Household Size`) %>%
  rename(fips = COUNTY, name = NAME, totp = `Total Population`,
         medage = `Median Age`, avghhsize = `Average Household Size`)
```

We need to combine this with our quasi-spatial county dataset. We can merge these two datasets on the county name, or better yet, the FIPS code. Let's do it:

```
analysisData <- left_join(countyDemogs, countyWells, by = "fips") %>%
  mutate(ones = 1)
analysisData
```

```
## Source: local data frame [67 x 8]
##
##     fips             name    totp medage avghhsize    id wells
##    (chr)            (chr)   (int)  (dbl)     (dbl) (chr) (dbl)
## 1    001     Adams County  101407   41.3      2.56     0     0
## 2    003 Allegheny County 1223348   41.3      2.23    54    30
## 3    005 Armstrong County   68941   44.5      2.38    33   172
## 4    007    Beaver County  170539   44.4      2.34     9    29
## 5    009   Bedford County   49762   43.9      2.43    39     1
## 6    011     Berks County  411442   39.1      2.59    41     0
## 7    013     Blair County  127089   42.0      2.37    32     6
## 8    015  Bradford County   62622   43.4      2.45    40  1189
## 9    017     Bucks County  625249   42.0      2.63    66     0
## 10   019    Butler County  183862   41.5      2.45    47   227
## ..   ...              ...     ...    ...       ...   ...   ...
## Variables not shown: ones (dbl)
```

Wahoo! We can *finally* run a regression!

```
myReg <- OLS(analysisData, "wells", c("ones", "totp", "medage", "avghhsize"))
xtable(myReg)
```

```
## % latex table generated in R 3.2.3 by xtable 1.8-2 package
## % Tue May 03 16:41:55 2016
## \begin{table}[ht]
## \centering
## \begin{tabular}{rlrrrr}
##   \hline
##  & variable & betahat & SE & tStat & tStatPVal \\
##   \hline
## 1 & ones & -96.53 & 1016.66 & -0.09 & 0.92 \\
##   2 & totp & -0.00 & 0.00 & -0.66 & 0.51 \\
##   3 & medage & 8.51 & 11.78 & 0.72 & 0.47 \\
##   4 & avghhsize & -58.08 & 296.57 & -0.20 & 0.85 \\
##    \hline
## \end{tabular}
## \end{table}
```

	variable	betahat	SE	tStat	tStatPVal
1	ones	4168.96	5093.60	0.82	0.42
2	totp	-0.00	0.00	-0.58	0.56
3	medage	24.29	59.04	0.41	0.68
4	avghhsize	-1879.07	1485.84	-1.26	0.21

Aaaaand nothing is statistically significant. That's a little anticlimatic.

Pointilism

Let's bring in one last dataset - nighttime lights.[14] This dataset grids up the world into approximately 1×1 km squares, and records a luminosity value, which is a measure of the amount of light eminating out of each pixel.[15] This dataset is available annually; I've grabbed the 2012 version to play with here. The original file was a TIFF image, which makes this dataset an *raster*: a giant gridded image. The original file also took up more than 1GB of space, so I've trimmed it down to just Pennsylvania, and exported it as an ASCII text file. Let's go ahead and bring it into `R`, and make sure that `R` knows that we're treating it as a raster.

```
# make the raster layer
lights <- readAsciiGrid("palights.txt") %>%
  raster()

# create a database version
lightsDF <- readAsciiGrid("palights.txt") %>%
  as.tbl_df()
names(lightsDF) <- c("dn", "long", "lat")
```

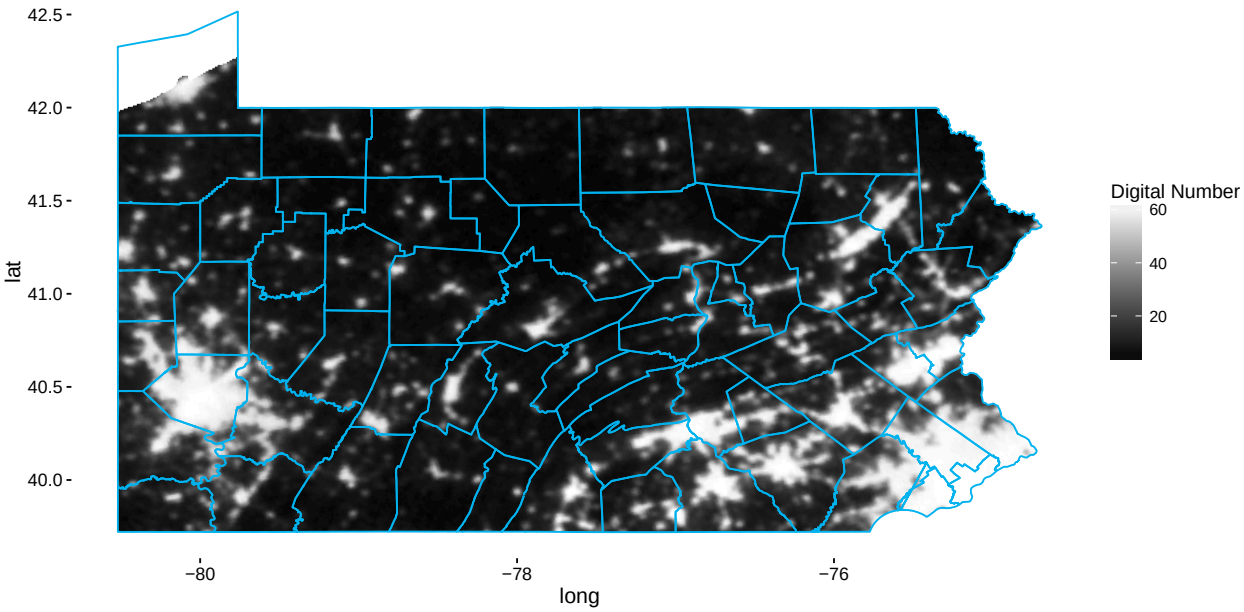
And plot:

```
lightsPlot <- ggplot(data = lightsDF, aes(x = long, y = lat)) +
  geom_raster(aes(fill = dn)) +
  geom_polygon(data = paCounties,
               aes(x = long, y = lat, group = id), color = 'deepskyblue2', fill = "NA") +
    myMapThemeStuff + labs(fill = "Digital Number") +
    scale_fill_gradient(low = "black", high = "white")
```

[14] I pre-processed this so that we only get the Pennsylvania lights, rather than the biggest file ever. The raw data files are available here.

[15] These data are becoming incredibly widely used by economists - often as a proxy for GDP. If you're interested in using them in your own research, please come talk to me. I've been using the data extensively, and love to share some insights and important things to watch out for with you.

```
lightsPlot
```



These two shapefiles line up pretttty well. The next thing we want to do is to calculate the average lights value for each county (this might take a little while):

```
countyLights <- extract(lights, counties, fun = mean, na.rm = T, df = T) %>%
  as.tbl_df()
countyLights <- mutate(countyLights, id = as.character(ID)) %>%
  select(palights.txt, id)

names(countyLights) <- c("dn", "id")
```

And merge into our county data:

```
analysisData <- left_join(analysisData, countyLights, by = "id") %>%
  na.omit()
analysisData

## Source: local data frame [66 x 9]
##
##     fips             name    totp medage avghhsize    id wells
##    (chr)            (chr)   (int)  (dbl)     (dbl) (chr) (dbl)
## 1    003 Allegheny County 1223348   41.3      2.23    54    30
## 2    005 Armstrong County   68941   44.5      2.38    33   172
## 3    007    Beaver County  170539   44.4      2.34     9    29
## 4    009   Bedford County   49762   43.9      2.43    39     1
## 5    011     Berks County  411442   39.1      2.59    41     0
## 6    013     Blair County  127089   42.0      2.37    32     6
## 7    015  Bradford County   62622   43.4      2.45    40  1189
```

```
## 8    017     Bucks County  625249   42.0      2.63    66     0
## 9    019    Butler County  183862   41.5      2.45    47   227
## 10   021   Cambria County  143679   43.8      2.30    62     7
## ..   ...              ...     ...    ...       ...   ...   ...
## Variables not shown: ones (dbl), dn (dbl)
```

And again, we can run our regression:

```
myReg2 <- OLS(analysisData, "wells", c("ones", "dn"))
myReg2

## Source: local data frame [2 x 5]
##
##   variable   betahat        SE    tStat tStatPVal
##     (fctr)     (dbl)     (dbl)    (dbl)     (dbl)
## 1     ones 52.468745 49.677617 1.056185 0.2948544
## 2       dn  3.045028  2.299625 1.324141 0.1901658

xtable(myReg2)

## % latex table generated in R 3.2.3 by xtable 1.8-2 package
## % Tue May 03 16:42:10 2016
## \begin{table}[ht]
## \centering
## \begin{tabular}{rlrrrr}
##   \hline
##  & variable & betahat & SE & tStat & tStatPVal \\
##   \hline
## 1 & ones & 52.47 & 49.68 & 1.06 & 0.29 \\
##   2 & dn & 3.05 & 2.30 & 1.32 & 0.19 \\
##    \hline
## \end{tabular}
## \end{table}
```

	variable	betahat	SE	tStat	tStatPVal
1	ones	760.86	254.08	2.99	0.00
2	dn	-10.51	11.76	-0.89	0.37

There doesn't appear to be a correlation between the number of wells drilled in Pennsylvaniand nighttime lights. Should we be worried? Not really - these datasets aren't perfect; the wells aren't all active anymore (and we haven't dealt with the time component of these data at all); there's not necessarily a clear reason to think that nighttime lights should be strongly related to fracking anyway (except for gas flaring - but that doesn't happen nearly as much in Pennsylvania as it does in North Dakota). Despite the lack of stars on this regression, I hope that this has been a useful exploration of the ways in which you can use `R` to manipulate spatial data.[16] We'll pick up next week with our very last section!

[16] As Betty once famously said, "This is not Hollywood - do not go looking for stars!"